

RAPPORT TECHNIQUE

Flask · Jenkins · Docker · Trivy · Kubernetes (Minikube)

Procédures d'installation, configuration et résolution d'erreurs
Du premier démarrage jusqu'au pipeline 100% fonctionnel

Auteur : Mara Sylvain LAHINIRIKO
Date : 26 February 2026
Environnement : Parrot OS · Docker · Minikube v1.38.1 · Jenkins LTS

Table des matières

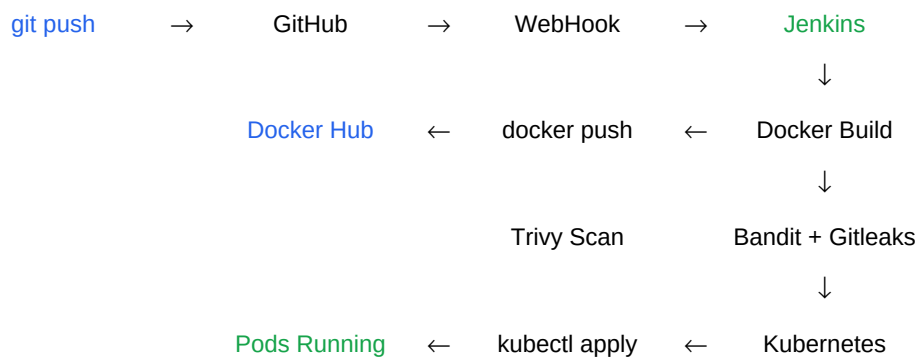
1. Architecture du projet
 2. Installation de Minikube
 3. Démarrage de Minikube — erreurs & corrections
 4. Configuration de Jenkins
 5. Connexion Jenkins ↔ Kubernetes
 6. Résolution de l'erreur de connexion réseau
 7. Génération du kubeconfig avec certificats embarqués
 8. Rendre la configuration permanente
 9. Pipeline Jenkins — exécution et erreurs
 10. Pipeline Jenkins — succès final
 11. Récapitulatif des commandes clés
-

1. Architecture du projet

Ce projet met en place un pipeline CI/CD complet intégrant des contrôles de sécurité à chaque étape. Voici le flux de traitement de bout en bout :

Composant	Rôle	Version
Jenkins (Docker)	Orchestrateur CI/CD	LTS 2.528.3
GitHub	Dépôt source + WebHook	—
Docker	Build & push des images	29.2.1
Docker Hub	Registry des images	—
Trivy	Scan CVE des images	0.69.1
Bandit	Analyse statique Python (SAST)	1.9.3
Gitleaks	Détection de secrets exposés	8.18.4
Minikube	Cluster Kubernetes local	1.38.1
Kubernetes	Orchestration des conteneurs	1.35.1
Flask (Alpine)	Application web Python	3.0.3

Flux du pipeline CI/CD :



2. Installation de Minikube

Minikube permet de faire tourner un cluster Kubernetes mono-nœud en local à l'intérieur d'un conteneur Docker. Il est utilisé comme environnement de déploiement cible pour notre pipeline CI/CD.

Pré-requis

Docker doit être installé et le démon Docker doit être actif. Minikube utilise Docker comme driver pour créer le nœud Kubernetes.

2.1 Téléchargement et installation du binaire

La commande curl télécharge le binaire Minikube pour Linux x86-64. sudo install le place dans /usr/local/bin avec les permissions correctes.

```
# Télécharger le binaire Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

# Installer dans /usr/local/bin
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Vérifier l'installation
minikube version
```

2.2 Démarrage du cluster

Minikube démarre un conteneur Docker qui simule un nœud Kubernetes complet (control-plane + worker). Le pilote docker est le plus adapté pour un environnement de développement local.

```
minikube start --driver=docker

# Résultat attendu :
# ■ minikube v1.38.1 sur Debian 13.1
# ■ Utilisation du pilote docker
# ■ Création de docker container (CPU=2, Memory=3072Mo) ...
# ■ Préparation de Kubernetes v1.35.1 sur Docker 29.2.1...
# ■ Terminé ! kubectl est maintenant configuré.
```

2.3 Installation de kubectl

kubectl est l'outil CLI pour interagir avec le cluster Kubernetes. Si le binaire n'est pas disponible directement, Minikube intègre sa propre version accessible via un alias.

```
# Option 1 : Téléchargement direct
curl -LO "https://dl.k8s.io/release/v1.30.0/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Option 2 : Utiliser kubectl intégré à Minikube (plus simple)
echo "alias kubectl='minikube kubectl --'" >> ~/.bashrc
source ~/.bashrc

# Vérifier
kubectl get nodes
```

Résultat obtenu

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	7m42s	v1.35.1

3. Démarrage de Minikube — erreurs & corrections

■ Étape 3.1 — Erreur : Minikube n'a jamais tourné

Symptôme

```
$ minikube status
E0226 12:15:23.223711 15484 status.go:178] status error: host: state: unknown state
"minikube"
```

Cause

Minikube n'a jamais été démarré ou le conteneur Docker qui le représente n'existe pas encore.

Solution

```
minikube start

# Si Minikube était déjà présent mais cassé, le supprimer d'abord :
minikube delete --all --purge
minikube start
```

■ Étape 3.2 — Erreur : IP déjà utilisée

Symptôme

```
■ Fermeture en raison de GUEST_PROVISION : error provisioning guest:
Failed to start host: recreate: creating host: create: creating:
create kic node: create container: can't create with that IP,
address already in use
```

Cause

L'IP 192.168.49.2 est déjà assignée à un autre conteneur Docker. L'ancien conteneur Minikube est resté bloqué.

Solution

```
# Supprimer le réseau et le conteneur bloqués
docker rm -f minikube 2>/dev/null || true
docker network rm minikube 2>/dev/null || true

# Nettoyer complètement Minikube
minikube delete --all --purge

# Redémarrer
minikube start
```

■ Étape 3.3 — Erreur : kubelet arrêté (NodeStatusUnknown)

Symptôme

```
$ kubectl describe node minikube | grep -A 5 'Conditions:'  
Ready Unknown ... NodeStatusUnknown Kubelet stopped posting node status.
```

Cause

Le processus kubelet à l'intérieur du conteneur Minikube s'est arrêté. Cela arrive souvent après une mise en veille du système ou un manque de ressources.

Solution

```
minikube stop  
minikube start  
  
# Si le problème persiste, augmenter les ressources :  
minikube delete  
minikube start --driver=docker --memory=4096 --cpus=2  
  
# Vérifier après ~1 minute  
kubectl get nodes # attendu : STATUS = Ready
```

■ Étape 3.4 — Erreur : kubeconfig mal configuré après redémarrage

Symptôme

```
$ minikube status  
kubeconfig: Misconfigured  
WARNING: Your kubectl is pointing to stale minikube-vm.  
To fix the kubectl context, run `minikube update-context`
```

Solution

```
minikube update-context  
  
# Résultat attendu :  
# ■ Le contexte "minikube" a été mis à jour pour pointer vers 192.168.49.2:8443
```

4. Configuration de Jenkins

Jenkins est exécuté dans un conteneur Docker. Pour que le pipeline CI/CD fonctionne, Jenkins a besoin d'accéder à Docker (pour builder les images), à kubectl (pour déployer sur Kubernetes), à Trivy et à Bandit.

4.1 Volumes et binds du conteneur Jenkins

Le conteneur Jenkins est lancé avec des bind mounts permettant le partage de ressources avec l'hôte. Le bind `/home/lms/.kube:/root/.kube` est particulièrement important : il rend la configuration Kubernetes automatiquement disponible dans Jenkins.

```
docker run -d \  
--name jenkins \  
-p 8080:8080 -p 50000:50000 \  
-v /home/lms/.kube:/root/.kube \  
-v /usr/local/bin/kubectl:/usr/local/bin/kubectl \  
-v /var/run/docker.sock:/var/run/docker.sock \  
-v "jenkins_home_backup:/var/jenkins_home" \  
jenkins/jenkins:ls
```

Mounts identifiés dans docker inspect

`/home/lms/.kube` → `/root/.kube` (kubeconfig, persistant) `/usr/local/bin/kubectl` → `/usr/local/bin/kubectl` (binaire kubectl) `/var/run/docker.sock` → `/var/run/docker.sock` (accès Docker) `jenkins_home_backup` → `/var/jenkins_home` (données Jenkins)

4.2 Installation des outils dans Jenkins

Une fois le conteneur démarré, les outils de sécurité doivent être installés à l'intérieur du conteneur. Ces installations sont persistantes via le volume `jenkins_home`.


```
# Installer Docker, Python, Bandit dans Jenkins
docker exec -u root jenkins bash -c "
apt-get update -qq && apt-get install -y docker.io python3 python3-pip -qq
pip3 install bandit --break-system-packages
ln -sf $(find /usr -name bandit -type f 2>/dev/null | head -1) /usr/local/bin/bandit
"

# Installer Trivy
docker exec -u root jenkins bash -c "
curl -sfl
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
| sh -s -- -b /usr/local/bin
"

# Installer Gitleaks
docker exec -u root jenkins bash -c "
curl -sSfL https://github.com/gitleaks/gitleaks/releases/download/v8.18.4/\
gitleaks_8.18.4_linux_x64.tar.gz | tar -xz -C /usr/local/bin gitleaks
"
```

4.3 Vérification des outils

```
docker exec jenkins docker ps # ■ Docker accessible
docker exec jenkins bandit --version # ■ bandit 1.9.3
docker exec jenkins trivy --version # ■ Version: 0.69.1
docker exec -u root jenkins kubectl get nodes # ■ minikube Ready
```

5. Connexion Jenkins ↔ Kubernetes

La connexion entre Jenkins (conteneur Docker) et Minikube (autre conteneur Docker) nécessite une configuration réseau et un kubeconfig valide avec des certificats accessibles.

■ Étape 5.1 — Première tentative : connexion refusée

```
$ docker exec -u root jenkins kubectl get nodes
E0226 09:12:32 memcache.go:265] "Unhandled Error" err="couldn't get current server
API group list: Get \"https://192.168.49.2:8443/api?timeout=32s\": dial tcp
192.168.49.2:8443: connect: connection refused"
The connection to the server 192.168.49.2:8443 was refused
```

Analyse du problème

Jenkins tourne dans un réseau Docker isolé (bridge) tandis que Minikube tourne dans son propre réseau Docker nommé 'minikube'. Les deux conteneurs ne peuvent pas communiquer directement. De plus, le port 8443 de Minikube n'est bindé que sur 127.0.0.1.

```
# Diagnostic : ports de Minikube
$ docker port minikube
8443/tcp -> 127.0.0.1:32771 # ← bindé seulement sur localhost !

# Diagnostic : IPs
$ docker inspect minikube | grep IPAddress
"IPAddress": "192.168.49.2", # réseau minikube

$ docker inspect jenkins | grep IPAddress
"IPAddress": "172.17.0.2", # réseau bridge
"IPAddress": "172.18.0.2", # réseau jenkins_default
```

■ Étape 5.2 — Solution : connecter Jenkins au réseau Minikube

La solution est de connecter Jenkins au réseau Docker 'minikube' afin qu'il puisse atteindre directement l'IP 192.168.49.2 du nœud Minikube.

```
# Connecter Jenkins au réseau Minikube
docker network connect minikube jenkins

# Vérifier que Jenkins est maintenant dans le réseau minikube
docker inspect jenkins | grep -A5 '"minikube"'
# → IPAddress: 192.168.49.3

# Résultat :
$ docker exec -u root jenkins kubectl get nodes
NAME STATUS ROLES AGE VERSION
minikube Ready control-plane 47m v1.35.1 ■
```

Topologie réseau finale

Réseau 'minikube' (192.168.49.0/24) : • minikube : 192.168.49.2 (serveur K8s, port 8443) • jenkins : 192.168.49.3 (client kubectl) Communication directe sur le réseau Docker interne.

6. Résolution des erreurs de certificats

■ Étape 6.1 — Erreur : certificats introuvables

Symptôme

```
$ docker exec -u root jenkins kubectl get nodes
Error in configuration:
* unable to read client-cert /home/lms/.minikube/profiles/minikube/client.crt
for minikube due to open ../client.crt: no such file or directory
* unable to read client-key /home/lms/.minikube/profiles/minikube/client.key
* unable to read certificate-authority /home/lms/.minikube/ca.crt
```

Cause

Le kubeconfig contient des chemins vers des fichiers de certificats sur la machine hôte (/home/lms/.minikube/). Ces fichiers n'existent pas dans le conteneur Jenkins.

Solution A : Copier les certificats dans Jenkins

```
docker exec -u root jenkins mkdir -p /home/lms/.minikube/profiles/minikube

docker cp ~/.minikube/ca.crt jenkins:/home/lms/.minikube/ca.crt
docker cp ~/.minikube/profiles/minikube/client.crt \
jenkins:/home/lms/.minikube/profiles/minikube/client.crt
docker cp ~/.minikube/profiles/minikube/client.key \
jenkins:/home/lms/.minikube/profiles/minikube/client.key

docker exec -u root jenkins chmod -R 644 /home/lms/.minikube/
```

Solution B (recommandée) : Embarquer les certificats en base64

La solution la plus propre est d'utiliser `kubectl config view --flatten --minify` qui génère un kubeconfig autonome avec tous les certificats encodés en base64 directement dans le fichier. Aucun fichier externe n'est nécessaire.

```
# Générer un kubeconfig avec certs embarqués
kubectl config view --flatten --minify > /tmp/kube-flat.yaml

# Vérifier qu'il contient bien 'certificate-authority-data' (pas un chemin)
grep 'certificate-authority-data' /tmp/kube-flat.yaml
# → certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0...

# Le kubeconfig est monté via le bind /home/lms/.kube → /root/.kube
# Il suffit de mettre à jour le fichier sur l'hôte :
cp /tmp/kube-flat.yaml ~/.kube/config

# Tester Jenkins
docker exec jenkins kubectl get nodes
# NAME STATUS ROLES AGE VERSION
# minikube Ready control-plane 23m v1.35.1 ■
```

7. Génération du kubeconfig avec certificats embarqués

Pour éviter de devoir reconfigurer kubectl à chaque redémarrage, la solution finale combine le bind mount `~/.kube` → `/root/.kube` avec un kubeconfig autonome (`--flatten`). Le fichier kubeconfig sur l'hôte EST le kubeconfig de Jenkins.

Problème : fichier kubeconfig vide après `>`

```
# ■ Cette commande vide le fichier AVANT d'écrire dedans !
kubectl config view --flatten --minify > ~/.kube/config

# ■ Utiliser un fichier temporaire intermédiaire :
kubectl config view --flatten --minify > /tmp/kube-flat.yaml
cp /tmp/kube-flat.yaml ~/.kube/config
```

Procédure complète après chaque redémarrage de Minikube

```
# 1. Remettre le contexte à jour
minikube update-context

# 2. Générer le kubeconfig autonome
kubectl config view --flatten --minify > /tmp/kube-flat.yaml

# 3. Appliquer
cp /tmp/kube-flat.yaml ~/.kube/config

# 4. Synchroniser pour l'utilisateur jenkins
docker exec -u root jenkins bash -c "
cp /root/.kube/config /var/jenkins_home/.kube/config
chown -R jenkins:jenkins /var/jenkins_home/.kube
chmod 644 /var/jenkins_home/.kube/config"

# 5. Test final
docker exec jenkins kubectl get nodes
```

Résultat final

NAME STATUS ROLES AGE VERSION minikube Ready control-plane 46m v1.35.1 ■ Jenkins (user jenkins, sans -u root) peut maintenant accéder au cluster.

8. Rendre la configuration permanente

Le bind mount `/home/lms/.kube → /root/.kube` garantit que le kubeconfig mis à jour sur l'hôte est automatiquement visible dans Jenkins. Pour automatiser la régénération après chaque démarrage du système, un service systemd utilisateur est créé.

8.1 Créer le script de régénération

```
cat > ~/fix-kube.sh << 'EOF'
#!/bin/bash
sleep 10 # Attendre que Docker et Minikube soient prêts
minikube update-context
kubectl config view --flatten --minify > /tmp/kube-flat.yaml
cp /tmp/kube-flat.yaml ~/.kube/config

# Synchroniser pour l'utilisateur jenkins
docker exec -u root jenkins bash -c "
cp /root/.kube/config /var/jenkins_home/.kube/config
chown -R jenkins:jenkins /var/jenkins_home/.kube
chmod 644 /var/jenkins_home/.kube/config" 2>/dev/null || true
EOF

chmod +x ~/fix-kube.sh
```

8.2 Créer le service systemd utilisateur

```
mkdir -p ~/.config/systemd/user

cat > ~/.config/systemd/user/fix-kube.service << 'EOF'
[Unit]
Description=Fix kubeconfig for Jenkins
After=docker.service

[Service]
Type=oneshot
ExecStart=/home/lms/fix-kube.sh

[Install]
WantedBy=default.target
EOF

systemctl --user daemon-reload
systemctl --user enable fix-kube.service
```

Statut du service

● fix-kube.service - Fix kubeconfig for Jenkins Loaded: loaded (/home/lms/.config/systemd/user/fix-kube.service; enabled) Active: inactive (dead) since Thu 2026-02-26 13:00:30 EAT Process: 30523 ExecStart=/home/lms/fix-kube.sh (code=exited, status=0/SUCCESS) ■ Le service s'exécutera automatiquement à chaque connexion utilisateur.

9. Pipeline Jenkins — erreurs rencontrées et corrections

■ Étape 9.1 — Option timestamps() invalide

Symptôme :

```
org.codehaus.groovy.control.MultipleCompilationErrorsException:  
WorkflowScript: 38: Type d'option non valide « timestamps ».  
Types d'options valides : [buildDiscarder, timeout, withCredentials, ...]
```

Cause :

Le plugin Timestamper n'est pas installé dans Jenkins. L'option timestamps() dans le bloc options{} nécessite ce plugin.

Correction :

```
# Supprimer timestamps() du Jenkinsfile  
# Avant :  
# options { timestamps() }  
  
# Après :  
# options { timeout(time: 30, unit: 'MINUTES') }  
  
git add Jenkinsfile  
git commit -m 'fix: remove timestamps() option - plugin not installed'  
git push
```

■ Étape 9.2 — docker: not found dans le pipeline

Symptôme :

```
docker: not found  
# Ou :  
/var/run/docker.sock: permission denied
```

Cause :

Le binaire Docker n'est pas installé dans le conteneur Jenkins, ou le socket Docker n'est pas accessible.

Correction :

```
docker exec -u root jenkins bash -c \  
'apt-get update -qq && apt-get install -y docker.io -qq  
chmod 666 /var/run/docker.sock'  
  
# Vérifier  
docker exec jenkins docker ps
```

■ Étape 9.3 — bandit: not found malgré pip install

Symptôme :

```
$ bandit --version  
bandit: command not found  
# même après pip3 install bandit
```

Cause :

pip installe les binaires dans ~/.local/bin qui n'est pas dans le PATH de l'environnement Jenkins. Le binaire existe mais n'est pas accessible.

Correction :

```
# Trouver le binaire et créer un symlink dans /usr/local/bin  
docker exec -u root jenkins bash -c "  
pip3 install bandit --break-system-packages  
BANDIT=$(find /usr /root /var/jenkins_home/.local -name bandit -type f | head -1)  
ln -sf $BANDIT /usr/local/bin/bandit  
"  
  
# Vérifier  
docker exec jenkins bandit --version # ■ bandit 1.9.3
```

■ Étape 9.4 — Trivy : &>/dev/null incompatible avec sh

Symptôme :

```
# Jenkinsfile utilise &>/dev/null (bash uniquement)  
# Jenkins exécute les scripts avec /bin/sh (dash)  
# Résultat : la condition if ! trivy... échoue toujours
```

Cause :

&>/dev/null est une syntaxe bash. Le shell par défaut de Jenkins est dash (sh), qui ne supporte pas cette syntaxe.

Correction :

```
# Remplacer toutes les occurrences dans le Jenkinsfile
# Avant :
if ! trivy --version &>/dev/null; then

# Après :
if ! trivy --version >/dev/null 2>&1; then

# Appliquer et pousser
git add Jenkinsfile
git commit -m 'fix: sh compatibility - replace &> with >/dev/null 2>&1'
git push
```

■ Étape 9.5 — requirements.txt manquant dans le repo

Symptôme :

```
COPY requirements.txt .
-----
ERROR: failed to solve: failed to read dockerfile: open requirements.txt:
no such file or directory
```

Cause :

Le fichier requirements.txt n'avait pas été commité dans le dépôt GitHub.

Correction :

```
cat > requirements.txt << 'EOF'
Flask==3.0.3
gunicorn==22.0.0
Werkzeug==3.0.3
EOF

git add requirements.txt
git commit -m 'fix: add missing requirements.txt'
git push
```

■ Étape 9.6 — CVE-2026-0861 bloque Trivy (image Debian)

Symptôme :

```
CRITICAL CVE-2026-0861 libc-bin / libc6
trivy image returned exit code 1
# → Pipeline bloqué à l'étape Trivy
```

Cause :

L'image de base python:3.12-slim (Debian Bookworm) contient une CVE HIGH dans la glibc. Aucun correctif Debian n'est disponible.

Correction :

```
# Solution : basculer vers Alpine (musl libc, non affectée)
# Modifier le Dockerfile :
FROM python:3.12-alpine AS builder

# Build résultant : 54.7MB (vs 125MB en slim)
# Trivy résultat : 0 CVE CRITICAL/HIGH ■

git add Dockerfile
git commit -m 'fix(security): switch to Alpine to fix CVE-2026-0861'
git push
```

■ Étape 9.7 — Docker Build timeout (apk add gcc prend >30 min)

Symptôme :

```
apk add gcc musl-dev libffi-dev # 1678 secondes pour 12/16 packages
# → Timeout de 30 minutes dépassé
```

Cause :

Flask, Gunicorn et Werkzeug ont des wheels Python précompilés sur PyPI. gcc et musl-dev ne sont pas nécessaires.

Correction :

```
# Supprimer les dépendances de compilation du Dockerfile
# Avant :
RUN apk add --no-cache gcc musl-dev libffi-dev

# Après : supprimer complètement cette ligne
# pip install trouvera les wheels précompilés automatiquement

git add Dockerfile
git commit -m 'fix: remove gcc from Alpine - pure Python packages dont need compilation'
git push
```

■ Étape 9.8 — Déploiement K8s : authentification refusée

Symptôme :

```
error: error validating k8s/deployment.yaml:  
failed to download openapi: <html>Authentication required</html>
```

Cause :

Le pipeline tourne en tant qu'utilisateur 'jenkins' (non root) qui n'a pas accès au kubeconfig dans /root/.kube/. De plus, le namespace 'production' n'existe pas encore.

Correction :

```
# 1. Créer le namespace production  
kubectl create namespace production  
  
# 2. Copier le kubeconfig pour l'utilisateur jenkins  
docker exec -u root jenkins bash -c "  
mkdir -p /var/jenkins_home/.kube  
cp /root/.kube/config /var/jenkins_home/.kube/config  
chown -R jenkins:jenkins /var/jenkins_home/.kube  
chmod 644 /var/jenkins_home/.kube/config"  
  
# 3. Vérifier  
docker exec jenkins kubectl get nodes # sans -u root
```

10. Pipeline Jenkins — succès final (Build #8)

Après résolution de toutes les erreurs, le pipeline s'est exécuté avec succès. Voici le récapitulatif des étapes et leur statut.

Étape	Statut	Détails
■ Clone Repository	■ Succès	Récupération du code depuis GitHub (branch main)
■ SAST - Bandit	■ Succès	Aucune vulnérabilité medium/high détectée dans le code Python
■ Docker Build	■ Succès	Image sylvain849/flask-cicd-app:8 (54.7 MB, Alpine 3.23.3)
■■ Trivy - Image Scan	■ Succès	0 CVE CRITICAL/HIGH sur Alpine — toutes les dépendances propres
■ Gitleaks - Secrets Scan	■ Succès	Aucun secret exposé détecté dans le code source
■ Docker Push	■ Succès	Image poussée sur Docker Hub : sylvain849/flask-cicd-app:8 + latest
■■ Trivy - Remote Scan	■ Succès	0 CVE confirmé sur l'image distante Docker Hub
■■ Deploy to Kubernetes	■ Succès	deployment 'flask-app' successfully rolled out (2 replicas)
■ Post-Deploy Verification	■ Succès	2 pods Running dans le namespace production

Extrait des logs du build réussi :

```
deployment.apps/flask-app created
service/flask-app-service created
networkpolicy.networking.k8s.io/flask-app-netpol created
deployment.apps/flask-app image updated
Waiting for deployment 'flask-app' rollout to finish...
deployment 'flask-app' successfully rolled out
■ Déploiement terminé avec succès

NAME READY STATUS RESTARTS AGE
flask-app-6d5df9f765-cbznb 1/1 Running 0 35s
flask-app-6d5df9f765-vmtq7 1/1 Running 0 104s

NAME TYPE CLUSTER-IP PORT(S) AGE
flask-app-service NodePort 10.110.210.240 80:30080/TCP 108s
```

■ PIPELINE RÉUSSI

Image : sylvain849/flask-cicd-app:8 · Build : #8

Accès à l'application déployée :

```
# Méthode 1 : URL automatique via Minikube
minikube service flask-app-service -n production

# Méthode 2 : Obtenir l'URL manuellement
minikube service flask-app-service -n production --url
# → http://192.168.49.2:30080

# Méthode 3 : Port-forward
kubectl port-forward service/flask-app-service 8081:80 -n production
# → http://localhost:8081
```

11. Récapitulatif des commandes clés

Ce tableau regroupe les commandes essentielles à mémoriser pour maintenir et opérer l'environnement CI/CD.

Minikube

Action	Commande
Démarrer	<code>minikube start</code>
Arrêter	<code>minikube stop</code>
Remettre le contexte	<code>minikube update-context</code>
Statut	<code>minikube status</code>
URL d'un service	<code>minikube service -n production --url</code>

kubeconfig (après restart)

Action	Commande
Régénérer	<code>kubectl config view --flatten --minify > /tmp/k.yaml</code>
Appliquer	<code>cp /tmp/k.yaml ~/.kube/config</code>
Sync jenkins	<code>docker exec -u root jenkins cp /root/.kube/config /var/jenkins_home/.kube/config</code>
Via alias	<code>fix-kube</code> (alias défini dans <code>~/.bashrc</code>)

Kubernetes

Action	Commande
Lister les pods	<code>kubectl get pods -n production</code>
Lister les services	<code>kubectl get services -n production</code>
Logs d'un pod	<code>kubectl logs -n production</code>
Créer namespace	<code>kubectl create namespace production</code>
Rolling update	<code>kubectl rollout status deployment/flask-app -n production</code>

Jenkins / Docker

Action	Commande
Test Docker	<code>docker exec jenkins docker ps</code>
Test kubectl	<code>docker exec jenkins kubectl get nodes</code>
Test Trivy	<code>docker exec jenkins trivy --version</code>
Test Bandit	<code>docker exec jenkins bandit --version</code>
Réseau minikube	<code>docker network connect minikube jenkins</code>

Rapport généré le 26/02/2026 à 11:02 · Projet CI/CD Pipeline Sécurisé · Mara Sylvain LAHINIRIKO